Netfilter mini-workshop

Pablo Neira Ayuso
<pablo@netfilter.org>

NetDev 0x14, Virtual
Aug 13th, 2020

Index

- Upstream updates since last netdev conference on each component of the Netfilter subsystem:
  - iptables
  - ipset
  - conntrack
  - IPVS
  - flowtable
  - nftables (*)
  - nftlb
- Discussion

iptables + ipset: updates

- iptables:
  - IDLETIMER target v1: alarm mode
    - “… continue to run even when the cpu is in suspended state”
  - 3 releases: 1.8.3 → 1.8.5
- ipset:
  - Perform garbage collection from workqueue to fix rcu detected stall in ipset hash set types
  - Wildcard support for the net,iface set
  - destination MAC address for hash:ip,mac sets
  - 4 releases: 7.2 → 7.6

conntrack: updates

- Bridge conntrack support
  - modprobe **nf_conntrack_bridge**
  - Register two hooks:
    - Bridge prerouting: nf_conntrack_in()
    - Bridge postrouting: nf_conntrack_confirm()
    - Defragments (don’t alter geometry, if possible)
  - Use **ct state** from the **bridge** family
  - Replaces **br_netfilter**
- Restore support for userspace conntrack helpers
- ctnetlink kernel side netlink dump filtering
- IPS_HW_OFFLOAD bit
- Enhance clash resolution to deal with DNS flow packets racing when setting up NAT

IPVS: updates

- On demand hook registration in IPVS
- Fallback to conntrack TCP tracker to handle connection reuse in IPVS
- Queue up delayed work to expire connections with no destination

flowtables: updates

- IPv6 support
- Hardware offl
- Fixes

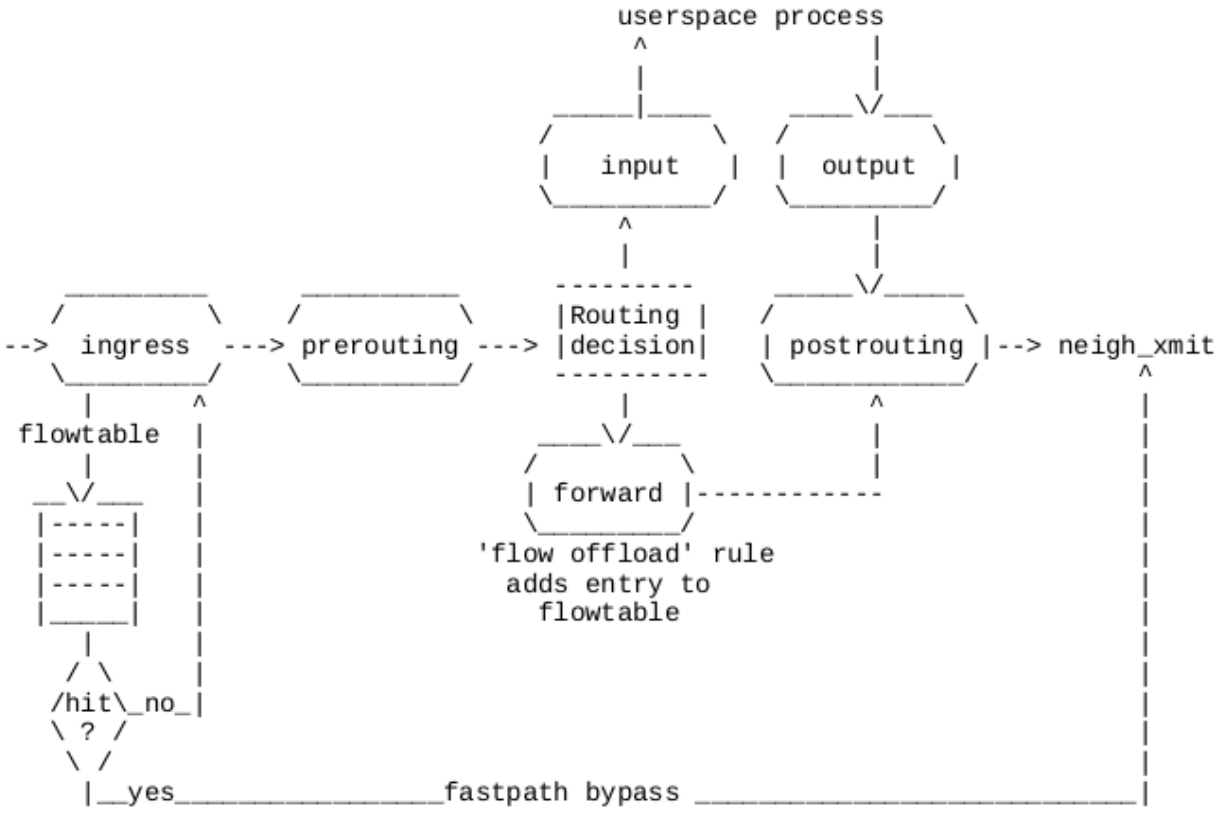


Fig.1 Netfilter hooks and flowtable interactions

nftables: updates

- 413 commits since March 2019 (last conf)
- 6 releases (0.9.1 → 0.9.6) *[+ upcoming 0.9.7]*
- 33 unique contributors
- Highlights:
  - New **features** (available up to Linux 5.9-rc)
  - Bugfixes
- … let’s show a quick summary

nftables: payload matching

- Transport header port matching
  - … ip protocol { tcp, udp } th dport 53 accept
  - … ip daddr . ip protocol . th dport @myset accept
- IPv4 options matching: lsrr, rr, ssrr and ra
  - … ip option rr exists drop
  - … ip option rr type 1 drop

nftables: meta

- time matching support
  - meta **time** \"2019-12-24 16:00\" **-** \"2020-01-02 7:00\" (in ISO format)
  - meta **hour** \"17:00\" **-** \"19:00\"
  - meta **day** \"Fri\"
- Secmark support
  - ct **secmark** set meta **secmark**
  - meta **secmark** set ct **secmark**
- Matching on bridge VLAN filtering metadata (bridge family only)
  - … meta **ibrpvid** 100
  - … meta **ibrvproto** vlan

nftables: sets and maps

- ranges in concatenations
  - table ip foo {
        set whitelist {
            type ipv4_addr . ipv4_addr . inet_service
            flags interval
            elements = { **192.168.10.35-192.168.10.40 . 192.68.11.123-192.168.11.125 . 80** }
        }
        chain bar {
            type filter hook prerouting priority filter; policy drop;
            ip saddr . ip daddr . tcp dport @whitelist accept
        }
    }

nftables: sets and maps

- typeof concatenations support for sets

```
table ip foo {
        set whitelist {
                typeof ip saddr . tcp dport
                elements = { 192.168.10.35 . 80,
192.168.10.101 . 80 }
        }
        chain bar {
                type filter hook prerouting priority filter;
policy drop;
                ip daddr . tcp dport @whitelist accept
        }
    }
```

nftables: sets and maps

- Restore expiration for set elements
  - add element ip x y { 1.1.1.1 timeout 30s **expires** 15s }
- Set element deletion from packet path:
  - ... delete @set5 { ip6 saddr . ip6 daddr }
- Set comments
  - add set x y { **comment \“this is my set\”\;** typeof ip saddr\; }
- -t/--terse option to exclude set elements from the listing

nftables: sets and maps

- Support for set counters:
  - table ip x {
    set y {
    typeof ip saddr
    counter
    elements = { 192.168.10.35, 192.168.10.101, 192.168.10.135 }

    chain z {
    type filter hook output priority filter; policy accept;
    ip daddr @y
    }
    }
- Support for restoring set counters

nftables: NAT

- NAT mappings with concatenations in rules
  - … dnat **ip addr . port** to ip saddr map { 1.1.1.1 : **2.2.2.2 . 443** }
- … also with named sets:
  - nft add map ip nat destinations { type ipv4_addr . inet_service : **ipv4_addr . inet_service** \; }
  - nft add rule ip nat pre dnat **ip addr . port** to ip saddr . tcp dport map @destinations

nftables: NAT

- Prefix and ranges in NAT
  - … iifname ens3 snat to **10.0.0.0/28**
  - … iifname ens3 snat to **10.0.0.1-10.0.0.15**
- Netmap support
  - table ip x {
    chain y {
    type nat hook postrouting priority srcnat; policy accept;
    snat ip **prefix** to ip saddr map { **10.141.11.0/24 : 192.168.2.0/24** }
    }
    }

nftables: NAT

- NAT intervals in maps
  - table ip x {
    map y {
    type ipv4_addr : **interval** ipv4_addr
    flags interval
    elements = { 10.141.10.0/24 : 192.168.2.2-192.168.2.4 }
    }
    chain y {
    type nat hook postrouting priority srcnat; policy accept;
    snat ip interval to ip saddr map @y
    }
    }

nftables: synproxy

- synproxy support (classic)
  - echo 0 > /proc/sys/net/netfilter/nf_conntrack_tcp_loose
  - table ip x {
        chain y {
            type filter hook prerouting priority raw; policy accept;
            tcp dport 8888 tcp flags syn notrack
        }
        chain z {
            type filter hook forward priority filter; policy accept;
            tcp dport 8888 ct state invalid,untracked synproxy **mss 1460 wscale 7 timestamp sack-perm**
            ct state invalid drop
    }
    }

nftables: synproxy

- Synproxy with maps
  - table ip foo {

```
        synproxy https-synproxy {
                mss 1460
               wscale 7
               timestamp sack-perm
        }
        synproxy other-synproxy {
                mss 1460
                wscale 5
        }
        chain pre {
                type filter hook prerouting priority raw; policy accept;
                tcp dport 8888 tcp flags syn notrack
        }
        chain bar {
                type filter hook forward priority filter; policy accept;
                ct state invalid,untracked synproxy name ip saddr map
{ 192.168.1.0/24 : "https-synproxy", 192.168.2.0/24 : "other-synproxy" }
        }
 }
```

nftables: expectations

- Custom conntrack expectations

```
table x {
        ct expectation myexpect {
                protocol tcp
                dport 5432
                timeout 1h
                size 12
                l3proto ip
        }
        chain input {
                type filter hook input priority 0;
                ct state new tcp dport 8888 ct expectation set myexpect
                ct state established,related counter accept
        }
}
```

nftables: reject

- Reject packets with 802.1q (from bridge family)
  - … ether type vlan reject with tcp reset
- Reject from prerouting
  - … add rule filter prerouting tcp dport 80 reject

nftables: reject

- Reject packets with 802.1q (from bridge family)
  - … ether type vlan reject with tcp reset
- Reject from prerouting
  - … add rule filter prerouting tcp dport 80 reject

nftables: netdev family

- Multidevice chain in netdev
  - add chain netdev x y { type filter hook ingress devices = **{ eth0, eth1 }** priority 0; }
- Hardware offload
  - Requires: ethtool -K eth0 hw-tc-offload on
  - table netdev x {
    chain y {
    type filter hook ingress device eth0 priority 10; **flags offload;**
    ip saddr 192.168.30.20 drop
    }
    }
- Matching on: packet header fields, input interface.
- Actions available are:
  - accept / drop action
  - Duplicate packet to port through `dup' and mirror packet to port through `fwd'.
- Chain priority 1 to 65535

nftables: flowtables

- Counters support for flowtables
  - table ip foo {
    flowtable bar {
    hook ingress priority -100
    devices = { eth0, eth1 }
    counter
    }
    chain forward {
    type filter hook forward priority filter;
    flow add @bar counter
    }
    }
  - You can list the counters via `conntrack -L'
- Support for updating flowtable devices

nftables: scripting

- variables in chain definitions
  - define **default_policy** = accept
    define **default_prio** = 0
    add chain ip foo bar { \
        type filter hook input priority **$default_prio**; policy **$default_policy** }
- Empty sets in variables

```
define BASE_ALLOWED_INCOMING_TCP_PORTS = {22, 80, 443}
define EXTRA_ALLOWED_INCOMING_TCP_PORTS = {}

table inet filter {
     chain input {
          type filter hook input priority 0; policy drop;
          tcp dport { $BASE_ALLOWED_INCOMING_TCP_PORTS,
$EXTRA_ALLOWED_INCOMING_TCP_PORTS }
     }
}
```

nftables: scripting

- variables in log prefix

```
define action = "DROP"
table x {
     chain y {
        ct state invalid log prefix "invalid $action:" drop
     }
}
```

- variables in chain device

```
define if_main = eth0
table netdev filter1 {
    chain ingress {
          type filter hook ingress device $if_main priority -500; policy accept;
     }
}
```

nftables: chain binding

- Implicit chain binding (Linux kernel >= 5.9-rc)

```
table inet x {
        chain y {
                type filter hook input priority 0;
                tcp dport 22 jump {
                        ip saddr { 127.0.0.0/8, 172.23.0.0/16, 192.168.13.0/24 } accept
                        ip6 saddr ::1/128 accept;
                }
        }
}
```

nftables: asorted

- # nft describe ipv4_addr
  datatype ipv4_addr (IPv4 address) (basetype integer), 32 bits
- Linenoise CLI support
  - ./configure --with-cli=linenoise

nftlb: Updates

- nftables load balancer
  - https://github.com/zevenet/nftlb/
- 2 releases: 0.5 → 0.6
- Highlights:
  - conntrack offload through flowtable
  - ingress support for farms
  - dual stack DSR and stateless NAT support
  - Improved backend health checks
  - Improved REST API

Discussion: new hooks?

- egress hook
- ifb ingress hook